KCC CODE CLUB · GAME MAKERS

Programme Overview

How the club runs, and why

Course: KCC Code Club: Game Makers
Ages: 7–12, no prior experience assumed
Length: 8 classes × 60 minutes, weekly, then a showcase event for families
Class size: 16 for the pilot term, 20 program ceiling thereafter
Volunteers: one per five students, never fewer than two
Cost: $40 for the eight-week term — $5 a session — collected up front

What the students will be able to do by week 8

Build a working arcade game from a blank project, on their own, and explain how it works. Underneath that: sequence, events, coordinates, loops, randomness, conditionals, boolean logic, sensing, variables, message passing, operators, and their own custom blocks. That is a real introduction to computer science, not a tour of a toy.

They will also have debugged their own code roughly two hundred times, which matters more than any of the above.

Staffing and class size

Size the class to the volunteers you actually have, not the other way round.

One volunteer per five students. Never fewer than two adults in the room, whatever the headcount. 16 students for the pilot term; 20 is the ceiling once the club has run once.

If the group skews young — more than about a third of them are 7 or 8 — work to 1:4 instead. A 7-year-old needs help reading the sheet, help dragging a block into a slot, and reassurance roughly three times as often as an 11-year-old, and a room that's half 7s and half 12s will feel understaffed at 1:5 even though the arithmetic says it isn't.

Students	Volunteers in the room	Spares left, from a team of 5
10	2	3
12	3	2
15	**3**	**2**
16	4	1
20	4	1

Fifteen is the sweet spot for a team of five. It's the largest class that still leaves two people in reserve, and reserves are what get you through eight weeks — somebody will be sick, travelling or dealing with a family thing, probably more than once. Sixteen is the cap rather than the target.

Why 20 is the ceiling, and why not in term one

The ratio governs how much *help* is available. It doesn't fix the three things that break in a big room:

- **The projector demo.** Ten minutes of live coding only works if the back row can read the blocks. Past about twenty, they can't.
- **Pace.** One lead can hold maybe twenty children through a build. Beyond that it isn't one class, it's two classes sharing a lead, badly.
- **Devices.** Twenty working laptops, twenty spare mice, and venue wi-fi that survives twenty simultaneous connections. Wi-fi that copes with fifteen and dies at twenty-five is extremely common — test at the number you're actually planning for. Families bring their own laptops, so the club controls none of this; at twenty, one forgotten charger is a child with nothing to do for the hour.

Hold 20 back until the club has run a full term, because at 20 you need four volunteers in the room every week and that leaves almost no slack. Prove the reliability of your team first.

If demand exceeds the cap, **run a second cohort rather than a bigger room**, and use the waitlist as the argument for it.

What to do with volunteers beyond the minimum

In rough order of value:

1. **Roster spares rather than cramming everyone in.** A briefed substitute who has read the plans is worth more than an extra pair of hands in week one. Keep at least two in reserve if you can.
2. **Zone the floaters.** Give each floating volunteer a couple of tables rather than the whole room. Unzoned floaters gravitate to whoever is loudest, and the quiet stuck child still gets missed — which was the whole point of the extra staffing.
3. **Split the class by age.** With enough volunteers for two rooms, running 7–9 and 10–12 separately is the single biggest quality improvement available to this programme. It removes the five-year spread, which is the hardest thing about a 7–12 room. It costs a second space, a second set of devices and double the prep — a term-two move unless the venue makes it easy.
4. **Rotate the lead.** Having two or three people who can run a session from the plan is what makes the club survive someone moving on.

A note on the pilot

Deliberate under-loading is the right call for a first term. Twelve to fifteen students with three volunteers and two spares will teach you what breaks, with enough slack to recover from it in the room. Sixteen with four and no reserve teaches you the same things much more painfully.

The class rhythm

Every class runs the same shape. Kids settle fastest into a predictable structure, and a predictable structure is also what lets a substitute volunteer run a session from the plan.

Time	Segment	Why it's there
0:00–0:05	Arrival and Show & Tell	Two students demo last week's homework. This is the single strongest motivator in the whole program and it costs nothing.

Time	Segment	Why it's there
0:05–0:15	New concept, live demo	You code on the projector and *deliberately make mistakes*. Watching an adult debug calmly is the lesson.
0:15–0:30	Class activity	One small, throwaway activity everyone does. Practising the concept with no risk to their real project.
0:30–0:55	Project build	Add this week's layer to their own game. The biggest block of the hour, on purpose.
0:55–1:00	Save, share, homework brief	Everyone saves and backs up. Nobody leaves with a broken game.

Ten minutes of talking, forty of hands-on. If you find yourself explaining for twenty minutes, stop and let them build. They learn it in their fingers, not their ears.

Two weeks bend the shape a little. Week 1 has a longer welcome instead of Show & Tell, and week 7 has two concepts, so its demo takes three minutes from the activity. Week 8 is its own thing: find the bug, playtest, polish, back up.

There's no break. An hour is a length a 7-year-old can sit through with a screen, and a break would come out of the project build. If a group is flagging, a one-minute stand-up stretch between the activity and the build costs almost nothing.

Why the project build gets the most time. Sky Catcher is the spine of the course. A child who misses a week's layer is behind for every week after it, while a child who misses the throwaway activity has lost nothing they need. So each week has one activity, not two, and the build gets the biggest share of the hour. The activities that didn't make the cut are still on the handouts, as bonus challenges.

The long project

One game, **Sky Catcher**, built across all eight weeks. Full detail in `long-project-sky-catcher.md`. Three design rules make it survive a volunteer program:

1. **It works at every checkpoint.** After every single class it is a playable, demo-able thing.
2. **Each week is one independent layer.** Miss week five and you can still do week six.
3. **A skeleton file exists at every stage.** A student who missed two weeks downloads the week-five skeleton, and they are back in the room. This is how you stop attrition compounding into dropout.

Students choose their own theme in week one — catching mangoes, stars, footballs, whatever they want. The mechanics are identical; the ownership is not.

Homework, bonus challenges, and the no-grading rule

Each week has **two homework tasks** and **three or four bonus challenges**.

Homework is short — fifteen to twenty minutes — and always applies something taught that day to their own game. Bonus challenges are open-ended ("make it do X"), deliberately harder, and exist to absorb the fast finishers who would otherwise be bored and disruptive.

Homework is never collected, never marked, and never chased. It is shown off voluntarily in Show & Tell. A child who does no homework all term and still turns up every week is a success. A child who feels bad about

unfinished homework and stops coming is the failure mode you are designing against.

Differentiation, in practice

You will have a 7-year-old who has never used a mouse properly next to a 12-year-old who has been on Scratch for a year. That five-year spread is the widest thing in the programme and it is what's most likely to go wrong. Handle it like this:

- The class activity is the floor. Everyone does it.
- Bonus challenges are the ceiling. Nobody is expected to finish them.
- Pair the experienced kid with a beginner from week two and give them the title of "helper". Being asked to explain is the fastest way to actually learn, and it converts a potential disruption into a second floating volunteer.
- If someone is genuinely far ahead, hand them the Raspberry Pi Foundation More Scratch pathway and let them run. Do not hold them to the class pace.

The 7-year-olds specifically

Dropping the floor from 8 to 7 is a bigger change than it looks, and it lands in two places.

Reading. The student handouts assume a competent reader. A 7-year-old often isn't one yet, and a child who can't read the sheet will sit quietly and do nothing rather than admit it. Read the activity steps aloud at the start of every activity block — all of them, every week, not just when someone asks. It costs ninety seconds and it removes the problem entirely.

Typing and mouse control. Dragging a block into a small slot is fine motor work, and it is genuinely harder at 7 than at 10. Spare USB mice stop being a nice-to-have and become essential. Seat the youngest students where the floating volunteer passes most often.

If you end up with three or more 7-year-olds, seat them together rather than scattering them. It's easier to give one table a slower pace than to chase four children around the room, and it saves them from measuring themselves against a 12-year-old on every task.

What to cut when you run out of time

You will run out of time. Roughly every second week. An hour has very little slack, so cut in this order:

1. The class activity — stop it at the quarter-hour mark wherever they've got to. It's throwaway by design.
2. The optional steps of the project build (each week's plan says which ones they are) — push them into homework and open the next class with them.

Never cut the core of the project build, Show & Tell, or the save-and-back-up at the end.

The honest cost of the hour is that a bad week now eats into the project build, and whatever spills over lands in homework — which the children who don't do homework never catch up on. The skeleton files are the fix: a child who fell behind in class starts next week from the skeleton, not from their half-finished game.

Attendance and attrition

Expect to lose two or three students over eight weeks; that is normal and not your fault. What is your fault is losing them because they fell behind. The skeleton files exist for exactly this. If a student misses a week,

message the parent with the skeleton link and one sentence on what they missed, and they will be fine.

Free programs get flaky attendance. A fee collected up front measurably improves it — people show up for what they have paid for. We charge **$40 for the term, which is $5 a session**: enough that a family has decided to come, low enough that it turns nobody away. Collect it before week one.

Agree with the committee, before you publish the fee, that no child is turned away over it and who is allowed to waive it quietly. A waiver that needs a meeting is a waiver nobody asks for.

The showcase

The showcase doesn't fit in a sixty-minute class. Demos run two minutes each plus about seventeen minutes of welcome, certificates and open play, so fifteen students need about fifty minutes, and week eight's own hour is spent getting the games finished. Run it as its own event: straight after week eight if the venue will give you the extra hour, or at the next KCC gathering, where the audience will be bigger anyway. Book it when you book the term, and tell families the date in the first letter. Details in the Showcase Run Sheet.

Success measures

Do not measure test scores; there is no test. Measure these:

- **Completion rate.** How many of the students who started week one demo at the showcase? Aim above 75%.
- **Voluntary homework rate.** How many bring something to Show & Tell? Aim above half by week four.
- **Week nine.** How many are still opening Scratch a month after the course ended? This is the only measure that actually matters.